Architettura dei sistemi di elaborazione (1)

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

alesandu0@gmail.com

Indice

1. Teoria
   - Sistemi di numerazione e basi posizionali.
   - Relazione tra livelli logici e segnali analogici.
2. Conversione Base Generica -> Base 10
   - Algoritmo della notazione polinomiale.
3. Conversione Base 10 -> Base Generica
   - Metodo delle divisioni successive.
4. Basi potenza l'una dell'altra
   - Conversione Base 2 -> Base 8 e Base 8 -> Base 2.
   - Conversione Base 2 -> Base 16 e Base 16 -> Base 2.
   - Caso Diverso: Base 3 -> Base 9.
5. Hamming

   - Calcolo bit parità e controllo su sequenza ricevuta

1. I sistemi di numerazione

- I sistemi di numerazione nascono per rappresentare i numeri.

- Un sistema numerico è quindi un linguaggio per scrivere i numeri e, come tale, si compone di:

- **Alfabeto**: l'insieme di simboli.

- **Grammatica**: le regole con le quali comporre sequenze valide (sintassi) e quelle per dedurne il significato numerico (semantica).

Sistema di numerazione romano

- È un sistema di **numerazione additivo**, ovvero a ogni simbolo è associato un valore e il numero rappresentato è dato dalla somma dei valori dei simboli:
  - I = 1
  - V = 5
  - X = 10
  - L = 50
  - C = 100
  - D = 500
  - M = 1000
- Esempio: XVII (romano) = 17 (decimale)

il numero zero non si può scrivere!

Sistema di numerazione posizionale

- Il numero di simboli utilizzati da un sistema di numerazione posizionale si chiama **base**.

- Il valore di ogni simbolo (o cifra) dipende dalla posizione che occupa nel numero, ad ogni posizione corrisponde una potenza della base.

$$N_b = \sum_{0}^{k} {}_j\, c_j \quad b^j$$

Esempio:

$321_{10} = 3 \cdot 10^2 + 2 \cdot 10^1 + 1 \cdot 10^0 = 3 \cdot 100 + 2 \cdot 10 + 1 \cdot 1$

Sistemi di numerazione posizionali

Esempi	base	alfabeto
1011_2 10101111_2	2	{0,1}
1011_8 12375_8	8	{0,1,2,3,4,5,6,7}
1011_{10} 12375_{10}	10	{0,1,2,3,4,5,6,7,8,9}
1011_{16} 12375_H $1A3F_{16}$	16	{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F}

Esempi di rappresentazioni numeriche

Decimale (es. 2002_{10}):

cifre	2	0	0	2
potenze	10^3	10^2	10^1	10^0
valori	2000	0	0	2

Binario (es. $0111_2 = 7_{10}$):

cifre	0	1	1	1
potenze	2^3	2^2	2^1	2^0
valori	0	4	2	1

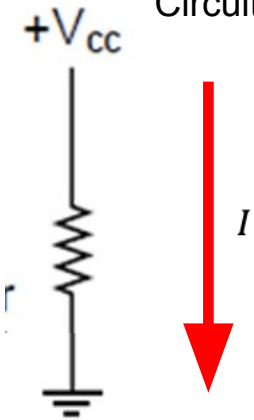
Analogia

Per capire l'elettricità, immaginiamo l'acqua che scorre in un tubo.

- **La Tensione (Volt - V):** È come la **pressione** dell'acqua. Rappresenta la differenza di potenziale: se un serbatoio è in alto e l'altro è in basso, l'acqua "spinge" per scendere. Senza differenza di altezza (potenziale), l'acqua non si muove.
- **La Corrente (Ampere - I):** È la **quantità** di acqua che scorre nel tubo in un secondo. È il flusso vero e proprio.
- **La Resistenza (Ohm - Omega):** È una **strizzatura** o un ostacolo nel tubo. Più il tubo è stretto, più è difficile per l'acqua passare.

La legge di Ohm mette in relazione questi tre elementi.

Differenza tra Circuito Aperto e Chiuso



Circuito chiuso

$$V = R \cdot I$$

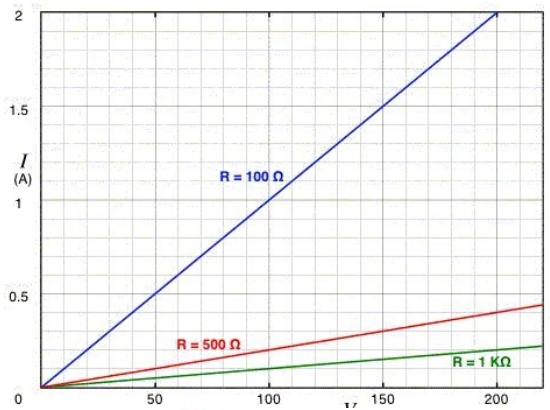
$$I = \frac{\Delta V}{R} = \frac{V_{cc} - 0}{R} = \frac{V_{cc}}{R}$$

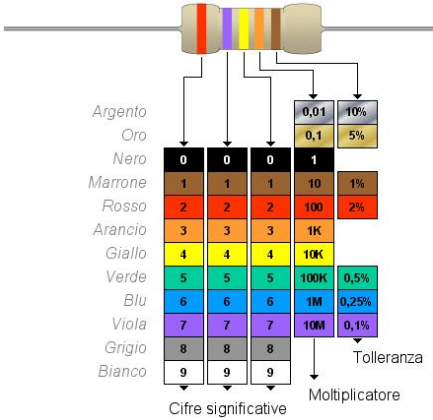
Circuito aperto

$$I = \frac{\Delta V}{R} = \frac{V_{cc} - V_{cc}}{R} = 0$$

Legge di Ohm

- **L'asse orizzontale (V):** Rappresenta la tensione (la "pressione" dell'acqua).
- **L'asse verticale (I):** Rappresenta la corrente (quanta acqua scorre).
- **Le linee colorate:** Rappresentano tre resistenze diverse.

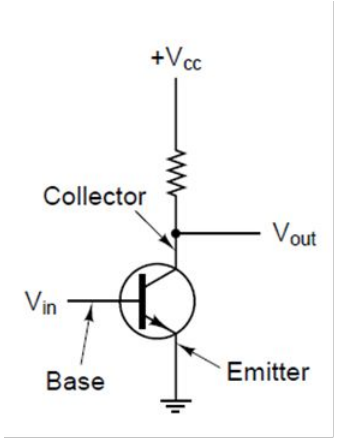


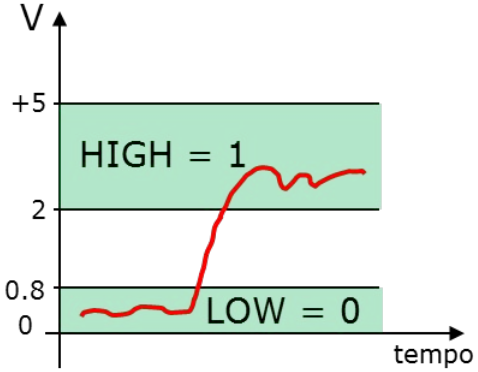


Segnali digitali e Legge di Ohm

I segnali elettrici analogici vengono interpretati come **livelli logici discreti** (0 e 1).

- **Il Transistor come Interruttore:** Il circuito a sinistra mostra un transistor in configurazione *Inverter*.
- **Logica TTL (Transistor-Transistor Logic):** Non esiste un singolo valore di tensione, ma degli **intervalli**:
  - **LOW (0):** Tensioni tra 0V e 0.8V.
  - **HIGH (1):** Tensioni tra 2V e 5V.
  - **Zona di indeterminazione:** L'intervallo 0.8V - 2.0V è non interpretato.





Livelli logici digitali dei TTL

2. Conversioni Base generica -> Base 10

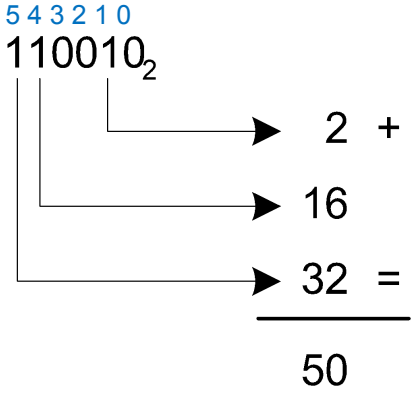
- Da qualsiasi base (b) a decimale:

$$N_b = \sum_{0}^{k} {}_j c_j \quad b^j$$

c_k	c_{k-1}	…	c_1	c_0
b^k	b^{k-1}	…	b^1	b^0

Metodo semplificato: Base 2 -> Base 10

- In una somma i bit a zero possono essere esclusi, possiamo sommare le sole potenze di due dove troviamo un uno:



Conversione: Base generica -> Base 10

- Trasformare $1F4_{16}$ in base 10.

cifre	**1**	**F**	**4**
pesi	16^2	16^1	16^0
valori	256	240	4

$$N_b = \sum_{0}^{k} {}_j c_j \quad b^j$$

$$1F4_{16} = 1 \cdot 16^2 + F \cdot 16^1 + 4 \cdot 16^0 = 1 \cdot 256 + 15 \cdot 16 + 4 \cdot 1 = 500_{10}$$

Conversione: Base generica -> Base 10

- Trasformare 314_5 in base 10.

cifre	**3**	**1**	**4**
pesi	5^2	5^1	5^0
valori	75	5	4

$$314_{16} = 3 \cdot 5^2 + 1 \cdot 5^1 + 4 \cdot 5^0 = 3 \cdot 25 + 1 \cdot 5 + 4 \cdot 1 = 84_{10}$$

Esercizio Conversione Base 16 -> Base 10

Converti il seguente numero in decimale

		$_{10}$
D	A	4 $_{16}$

Esercizio Conversione Base 16 -> Base 10

Converti il seguente numero in decimale

$4*16^0 + 10*16^1 + 13*16^2 = 3492_{10}$		
D	A	4_{16}

3. Conversioni Base 10 -> Base generica

- Da decimale a qualsiasi base:

Metodo delle divisioni successive

Metodo semplificato: Base 10 -> Base 2

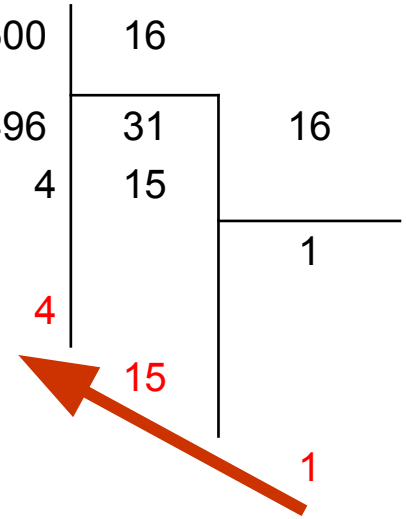
- Dividere per 2 corrisponde a stabilire se un numero è pari o dispari…

50	0
25	1
12	0
6	0
3	1
1	1
0	

$50_{10} = 110010_2$

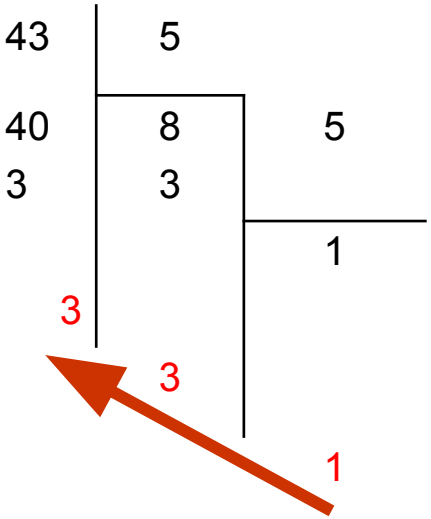
Conversione: Base 10 -> Base generica

- Trasformare 500_{10} in base 16.



Conversione: Base 10 -> Base generica

- Trasformare 43_{10} in base 5.



Esercizio Conversione Base 10 -> Base 2

Converti il seguente numero in binario

$$205_{10}$$

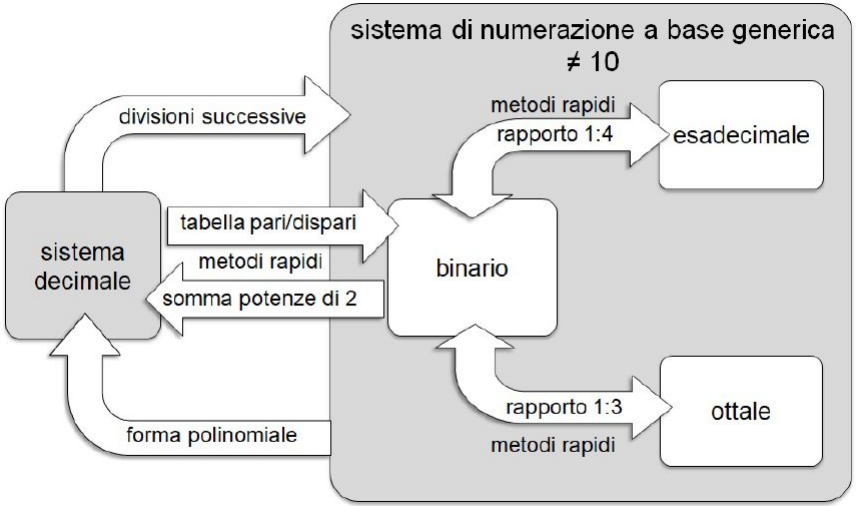
Esercizio Conversione Base 10 -> Base 2

Converti il seguente numero in binario

205	1
102	0
51	1
25	1
12	0
6	0
3	1
1	1
0	

11001101_2

Tabella riassuntiva



4. Basi potenza l'una dell'altra
Base 2 (2^1) <-> Base 8 (2^3)

- Essendo le basi 8 e 16 potenze di 2, esiste un procedimento semplice di trasformazione dall'una all'altra base:
- Base 2 -> base 8: si suddivide il numero binario in gruppi di 3 bit (a partire dalla cifra meno significativa) e si trasforma ogni gruppo nella corrispondente cifra ottale:

1	0	1	1	1	0	1	0	0
4	2	1	4	2	1	4	2	1

564_8

- Base 8 -> base 2: ad ogni cifra ottale corrispondono 3 bit, si usano le potenze di due (4 2 1) per comporre la cifra in ottale.

Base 2 (2^1) <-> Base 16 (2^4)

- Base 2 -> base 16: si suddivide il numero binario in gruppi di 4 bit (a partire dalla cifra meno significativa) e si trasforma ogni gruppo nella corrispondente cifra esadecimale:

0	0	0	1	1	1	1	1	0	1	0	0
8	4	2	1	8	4	2	1	8	4	2	1

$1F4_{16}$

- Base 16 -> base 2: ad ogni cifra esadecimale corrispondono 4 bit, si usano le potenze di due (8 4 2 1) per comporre la cifra in esadecimale.

Base 3 <-> Base 9 (3^2)

- Trasformare 2102_3 in base 9.

2	1	0	2
3	1	3	1

$$2102_3 = (2 \cdot 3^1 + 1 \cdot 3^0) \mid (0 \cdot 3^1 + 2 \cdot 3^0) = 72_9$$

Esercizio Conversione Base 9 -> Base 3

Converti il seguente numero in base 9

1	12	22_3
		$_9$

Esercizio Conversione Base 9 -> Base 3

Converti il seguente numero in base 9

1	12	22_3
1	5	8_9

$1*3^0 = 1$ | $1*3^1 + 2*3^0 = 5$ | $2*3^1 + 2*3^0 = 8$

Esercizio Conversione Base 2 -> Base 8

Converti il seguente numero in ottale

1	1	0	0	1	1	1	0	1_2

Esercizio Conversione Base 2 -> Base 8

Converti il seguente numero in ottale

6			3			5		$_8$
1	1	0	0	1	1	1	0	1_2

Esercizio Conversione Base 8 -> Base 2

Converti il seguente numero in binario

7	0	2_8

Esercizio Conversione Base 8 -> Base 2

Converti il seguente numero in binario

7			0			2		$_8$
1	1	1	0	0	0	0	1	0_2

Esercizio Conversione Base 8 -> Base 16

Converti il seguente numero in esadecimale

7	0	2 $_{8}$
		$_{16}$

Esercizio Conversione Base 8 -> Base 16

Converti il seguente numero in esadecimale

7			0			2		$_8$
1	1	1	0	0	0	0	1	0_2
								$_{16}$

Esercizio Conversione Base 8 -> Base 16

Converti il seguente numero in esadecimale

0			7			0			2		$_8$
0	0	0	1	1	1	0	0	0	0	1	0_2
1				C					2		$_{16}$

Hamming

In ogni sistema di comunicazione (Wi-Fi, Hard Disk, RAM), i dati viaggiano come bit (0 e 1). Tuttavia, il "rumore" elettromagnetico può invertire un bit.

- **Senza controllo:** Riceviamo un dato errato senza saperlo.
- **Bit di Parità semplice:** Ci dice *se* c'è un errore, ma non *dove*.
- **Codice di Hamming:** Non solo rileva l'errore, ma lo **corregge** individuando l'esatta posizione del bit corrotto.

Bit di parità

Il Codice di Hamming inserisce dei **bit di parità** (o bit di controllo) in posizioni strategiche (le potenze di 2: posizioni 1, 2, 4, 8).

Se un bit cambia durante la trasmissione, la combinazione dei risultati dei bit di controllo punterà verso il bit sbagliato.

$2^n >= m + n + 1$

Se hai **4 bit** di dati (m=4): $2^3 >= 4+3+1 = 8$. Servono **3 bit** di parità.
Se hai **11 bit** di dati (m=11): $2^4 >= 11+4+1 = 16$. Servono **4 bit** di parità.

Hamming Code: Bit di parità

Calcolare i bit di parità da aggiungere ad $1B_{16}$

1				B$_{16}$			
0	0	0	1	1	0	1	1_2

Hamming Code: Bit di parità

Calcolare i bit di parità da aggiungere ad $1B_{16}$

1				B$_{16}$			
0	0	0	1	1	0	1	1_2

$m = 8$

$n + m + 1 \leq 2^n$

$n = 4$

Hamming Code: Calcolo bit correzione d’errore

Calcola i bit di correzione d’errore da aggiungere alla parola $1B_{16}$

							b_3				b_2		b_1	b_0

Hamming Code: Calcolo bit correzione d’errore

Calcola i bit di correzione d’errore da aggiungere alla parola $1B_{16}$

12	11	10	9	8	7	6	5	4	3	2	1
0	0	0	1	b_3	1	0	1	b_2	1	b_1	b_0

Hamming Code: Calcolo bit correzione d’errore

Calcola i bit di correzione d’errore da aggiungere alla parola $1B_{16}$

12	11	10	9	8	7	6	5	4	3	2	1
0	0	0	1	b_3	1	0	1	b_2	1	b_1	b_0
	📌		📌		📌		📌		📌		👮‍♀️

Hamming Code: Calcolo bit correzione d’errore

Calcola i bit di correzione d’errore da aggiungere alla parola $1B_{16}$

12	11	10	9	8	7	6	5	4	3	2	1
0	0	0	1	b_3	1	0	1	b_2	1	b_1	b_0

Hamming Code: Calcolo bit correzione d’errore

Calcola i bit di correzione d’errore da aggiungere alla parola $1B_{16}$

12	11	10	9	8	7	6	5	4	3	2	1
0	0	0	1	b_3	1	0	1	b_2	1	b_1	b_0

Hamming Code: Calcolo bit correzione d’errore

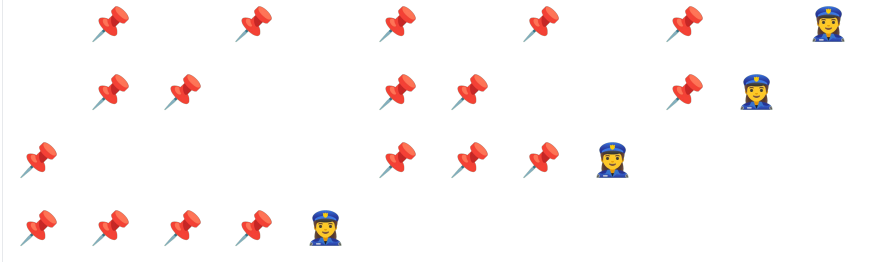
Calcola i bit di correzione d’errore da aggiungere alla parola $1B_{16}$

12	11	10	9	8	7	6	5	4	3	2	1
0	0	0	1	b_3	1	0	1	b_2	1	b_1	b_0

Hamming Code: Calcolo bit correzione d’errore

Calcola i bit di correzione d’errore da aggiungere alla parola $1B_{16}$

12	11	10	9	8	7	6	5	4	3	2	1
0	0	0	1	1	1	0	1	0	1	0	0



Hamming Code: Controllo bit parità

Assumendo abbiamo ricevuto la sequenza $2B1_{16}$ calcola i bit di parità

Hamming Code: Controllo bit parità

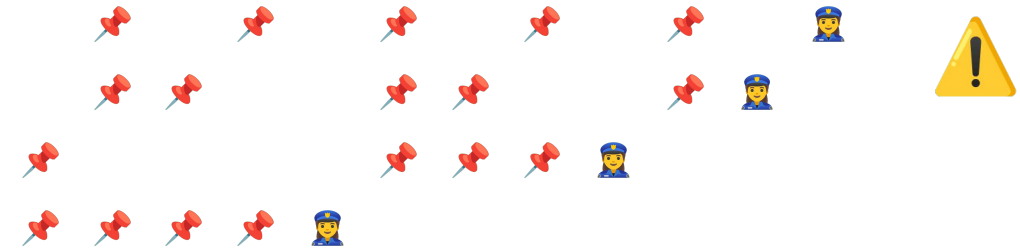
Assumendo abbiamo ricevuto la sequenza $2B1_{16}$ calcola i bit di parità

12	11	10	9	8	7	6	5	4	3	2	1
0	0	1	0	1	0	1	1	0	0	0	1

Hamming Code: Controllo bit parità

Assumendo abbiamo ricevuto la sequenza $2B1_{16}$ calcola i bit di parità

12	11	10	9	8	7	6	5	4	3	2	1
0	0	1	0	1	0	1	1	0	0	0	1



Hamming Code: Controllo bit parità

Assumendo abbiamo ricevuto la sequenza $2B1_{16}$ calcola i bit di parità

12	11	10	9	8	7	6	5	4	3	2	1
0	0	1	0	1	0	1	1	0	0	0	1

b_3	b_2	b_1	b_0
0	0	0	0

Hamming Code: Controllo bit parità

Assumendo abbiamo ricevuto la sequenza ~~2B1~~2B5$_{16}$ calcola i bit di parità

12	11	10	9	8	7	6	5	4	3	2	1
0	0	1	0	1	0	1	1	0	0	0	1

b_3	b_2	b_1	b_0
0	0	0	0

Hamming Code: Controllo bit parità

Assumendo abbiamo ricevuto la sequenza ~~2B1~~2B5$_{16}$ calcola i bit di parità

12	11	10	9	8	7	6	5	4	3	2	1
0	0	1	0	1	0	1	1	0	1	0	1

b_3	b_2	b_1	b_0
0	0	0	0

Hamming Code: Controllo bit parità

Assumendo abbiamo ricevuto la sequenza ~~2B1~~2B5$_{16}$ calcola i bit di parità

12	11	10	9	8	7	6	5	4	3	2	1
0	0	1	0	1	0	1	1	0	1	0	1

b_3	b_2	b_1	b_0
0	0	1	1

Esercizio Hamming

Assumendo abbiamo ricevuto la sequenza ~~B3~~A95$_{16}$ calcola i bit di parità

Esercizio Hamming

Assumendo abbiamo ricevuto la sequenza ~~B3~~A95$_{16}$ calcola i bit di parità

12	11	10	9	8	7	6	5	4	3	2	1
1	0	1	0	1	0	0	1	0	1	0	1
	📌		📌		📌		📌		📌		📌
	📌	📌			📌	📌			📌	📌	
📌					📌	📌	📌	📌			
📌	📌	📌	📌	📌							

⚠️

b_3	b_2	b_1	b_0
1	0	0	1

Esercizio Hamming

Assumendo abbiamo ricevuto la sequenza ~~FF~~E77$_{16}$ calcola i bit di parità

Esercizio Hamming

Assumendo abbiamo ricevuto la sequenza ~~FF~~E77$_{16}$ calcola i bit di parità

12	11	10	9	8	7	6	5	4	3	2	1
1	1	1	0	0	1	1	1	0	1	1	1

b_3	b_2	b_1	b_0
1	0	0	1

Esercizio Hamming

Assumendo abbiamo ricevuto la sequenza ~~11~~386_{16} calcola i bit di parità

Esercizio Hamming

Assumendo abbiamo ricevuto la sequenza ~~11~~386_{16} calcola i bit di parità

12	11	10	9	8	7	6	5	4	3	2	1
0	0	1	1	1	0	0	0	0	1	1	0

b_3	b_2	b_1	b_0
1	0	1	0